

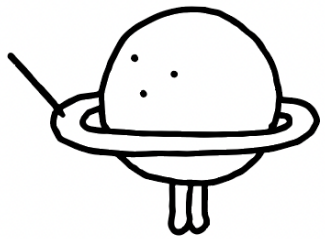
JFR 분석으로 EC2 CPU 스로틀링 해소하기 (EC2 CPU 19.5% → 4.6%, DB CPU 8.7% -> 5.4%)

우아한테크코스 BE 7기 메이

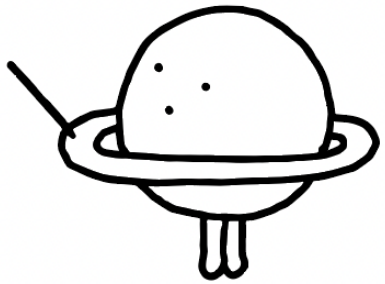


목차

1. 문제 상황
2. 원인 찾기
3. JFR 기반 문제 원인 특정
4. 개선
5. 결과



문제 상황
(feat.봄봄)

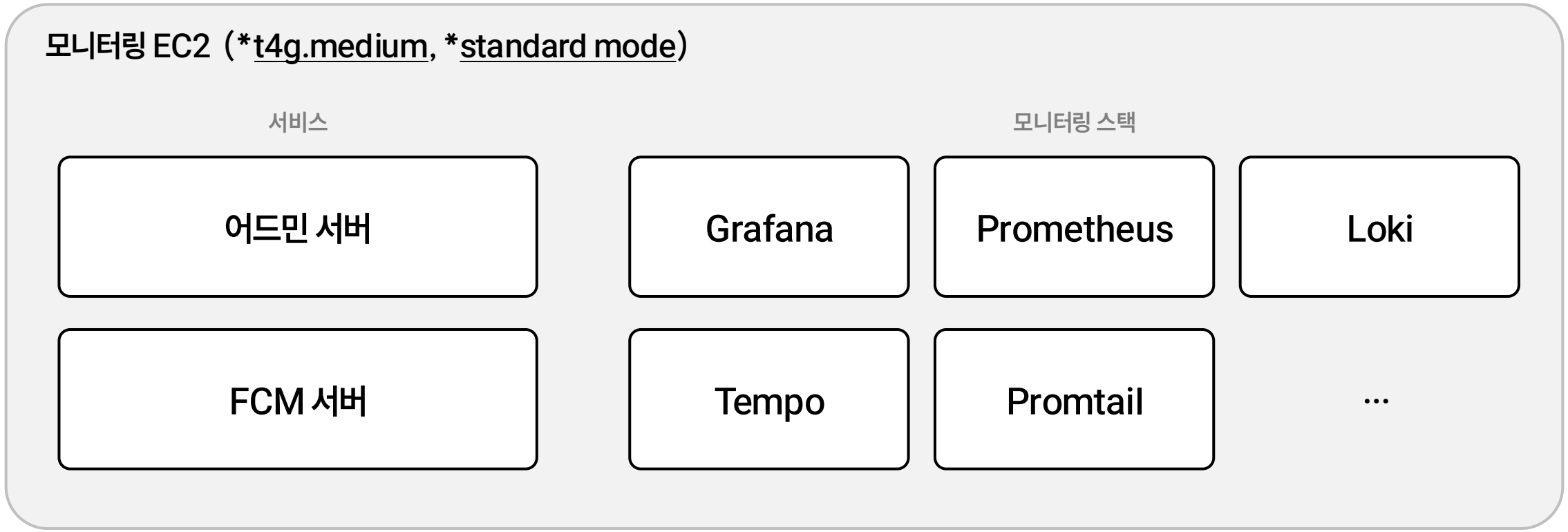


모니터링 EC2 CPU 스로틀링 발생



크레딧을 전부 소모하자 EC2의 vCPU가 베이스라인 20%대에 고정됨

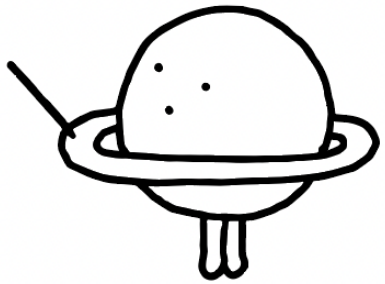
모니터링 EC2 구성



스로틀링에 의해 모니터링 EC2의 작업들 (모니터링 대시보드 조회, FCM 알림 전송 등)의 성능이 저하

※ 버스터블 인스턴스(t4g.medium): 평소엔 낮은 baseline만큼의 CPU를 보장하면서 남는 여유를 크레딧으로 적립해 두었다가, 필요할 때 그 크레딧으로 baseline 이상 버스트하는 저렴한 인스턴스
※ standard mode: 크레딧이 소진되면 CPU가 베이스라인 수준으로 제한됨

원인 찾기



높은 CPU 사용 프로세스 추측

가설 1 모니터링 관련, 메트릭 · 로그 · 트레이스 수집 통신 증가

→ I/O Bound니까 아닐 것 같음

가설 2 모니터링 관련 , Grafana alert rule 15개가 1분 단위로 쿼리 수행

→ 가능성 있음

가설 3 FCM 관련, 사용자 증가로 알림 발송량 증가

→ 가능성 있음

→ 어떤 프로세스에서 부하가 발생하는지 확인 필요

프로세스별 CPU 점유율 확인

```
ps aux --sort=-%cpu | head -20
```

```
[ec2-user@ip-10-0-5-182 bin]$ ps aux --sort=-%cpu | head -20
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
100         8504 10.2 11.0 2841252 435060 ?       Ssl  Jul06 11446:01 java -Duser.
root        6860  5.5  1.5 2654160 61932 ?       Ssl  Jul06 6251:24 /usr/bin/dock
root        8475  3.7  0.1 2229980 4884 ?        Sl   Jul06 4138:19 /usr/bin/cont
root        7901  2.8 10.1 28072908 396796 ?     Ssl  Jul06 3219:47 /bin/promethe
root        7880  2.3  5.3 2619256 209628 ?      Ssl  Jul06 2593:08 grafana serve
root        7974  1.1  1.1 2093016 46200 ?       Ssl  Jul06 1326:46 /usr/bin/prom
root        8083  0.9  3.8 2210136 152548 ?      Ssl  Jul06 1092:43 /usr/bin/loki
root     1795149  0.5 13.6 2364424 535744 ?      Ssl  00:54   1:24 java -Djava.se
```

FCM 서버 10% - 개선 대상으로 설정

모니터링(grafana, prometheus, …) 총 7.4%

어드민 서버 0.5%

스레드별 CPU 점유율 확인

FCM 프로세스 내 스레드별 CPU 사용률

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
8864	100	20	0	2841220	424884	9952	S	13.2	10.8	8779:41	scheduling-1
8700	100	20	0	2841220	424884	9952	S	2.3	10.8	1450:44	AsyncAppender-W
8504	100	20	0	2841220	424884	9952	S	0.0	10.8	0:00.03	java
8579	100	20	0	2841220	424884	9952	S	0.0	10.8	0:12.81	java
8610	100	20	0	2841220	424884	9952	S	0.0	10.8	164:38.52	GC Thread#0

알림 처리 scheduling 스레드가 전체 15% 중 13%를 차지

→ FCM 서버 안에서도 scheduling 스레드가 범인 후보

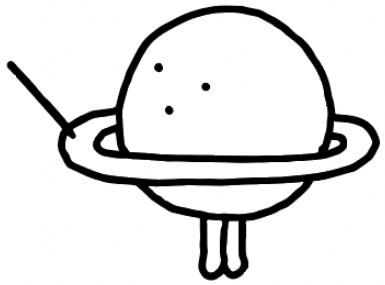
다음 단계

scheduling 스레드가 CPU를 오래 점유한다는 건 알겠는데,
그 중에서도 어떤 코드에서 오래 걸리는지 파악하고 싶었다



→ JFR (Java Flight Recorder)로 분석해보기

JFR 기반
문제 원인 특정





JFR(Java Flight Recorder)

JVM에 내장된 프로파일러 — 이벤트가 발생할 때마다 기록

instant

순간적으로 발생

예) 예외 발생

duration

시작~종료 시간이 있음

예) GC Pause, 락 대기, 소켓 읽기

periodic

일정 주기로 상태 기록

예) CPU, 힙 사용량, 스레드 수

sampling

주기적으로 스레드 스택 샘플링

현재 실행 중인 메서드 기록
→ 많이 찍힐수록 CPU를 많이 사용

→ 샘플링 기반으로 어떤 메서드가 오래 걸리는지 확인할 수 있다

JFR 분석 결과 – scheduling-1 스레드는 무엇을 하고 있었나

```
jcmd 1 JFR.start name=cpu_1h settings=profile duration=1h filename=/tmp/cpu-1h.jfr
```

1시간 기록 → 샘플 약 8,900개 중 7,856개가 scheduling-1 스레드 (88%)

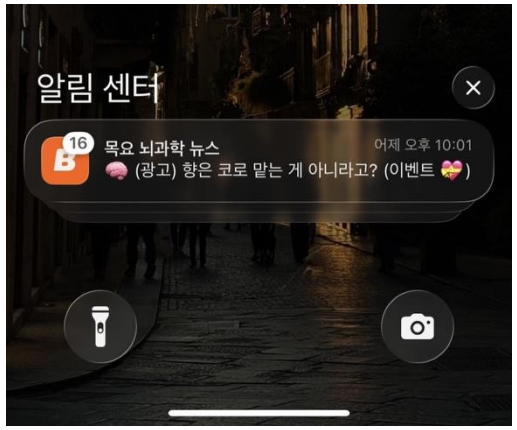
`scheduling-1` 스레드 CPU 샘플의 88%(7,856/8,900) 중 카테고리별 비중:

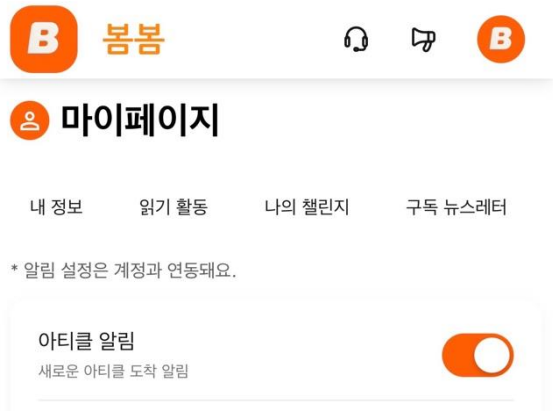
원인	비중
Hibernate/JPA (쿼리 준비/실행)	37.8%
JDBC/MySQL 드라이버 (`setAutoCommit`, `commit()` 반복)	21.6%
로깅 (Logback 동기 포매팅)	14.6%
문자열 빌딩 (로그 메시지 조립)	5.3%
리플렉션 (Micrometer 로그 메트릭 계측)	3.3%
FCM/Firebase 네트워크 호출	**0.02%** (2개 샘플뿐)

DB 접근
JPA + JDBC

시간 대부분을 DB 접근에 쓰고 있었다 (FCM 네트워크 호출은 0.02%)

JFR 분석 결과 – 가장 큰 샘플을 차지한 메서드





ArticleArrivalNotificationProcessor
.processPendingNotifications()

→

NotificationSettingService
.isEnabled()

→

DB
조회

아티클 도착 알림 (스케줄러)

알림 수신 여부 확인

전체 샘플의 약 60%가 아티클 알림 수신 거절 여부를 확인할 때 찍힘

코드 분석 – 아티클 도착 알림 스케줄러

```java
@Override
public void processPendingNotifications(LocalDateTime now) {
    List<ArticleArrivalNotification> pendingNotifications =
            notificationRepository.findRetryCandidates(
                    List.of(NotificationStatus.PENDING, NotificationStatus.FAILED),
                    now
            );

    log.info("[{}] 처리할 알림 개수: {}", type(), pendingNotifications.size());

    for (ArticleArrivalNotification notification : pendingNotifications) {
        try {
            if (!notification.shouldRetry()) {
                log.warn("[{}] 최대 재시도 횟수 초과: notificationId={}, attempts={}",
                        type(), notification.getId(), notification.getAttempts());
                articleStatusService.moveToFailedTableIfExceeded(notification);
                continue;
            }

            notificationProcessingService.processNotification(
                    notification,
                    NotificationCategory.ARTICLE,
                    articleStatusService
            );
        } catch (Exception e) {
            log.error("[{}] 알림 처리 중 오류 발생: notificationId={}", type(), notification.getId(), e);
        }
    }
}
```

30초 간격으로 실행
DB에서 PENDING · FAILED
알림을 모두 조회

조회한 알림마다
processNotification()으로
하나씩 처리

코드 분석 – 알림 하나를 처리하는 과정

각 알림별로 하나씩 처리

```java
@Transactional
public <T extends Notification> void processNotification(T notification,
                                                         NotificationCategory category,
                                                         NotificationStatusHandler<T> statusHandler) {
    Long memberId = notification.getMemberId();
    if (!notificationSettingService.isEnabled(memberId, category)) {
        log.info("알림 수신 동의하지 않음: memberId={}, category={}", memberId, category);
        return;
    }

    List<MemberFcmToken> fcmTokens = notificationTokenService.resolveTokens(memberId);
    if (fcmTokens.isEmpty()) {
        statusHandler.markAsFailed(notification, reason: "FCM 토큰 없음");
        return;
    }

    NotificationResultResponse result = notificationSender.sendToAllDevices(notification, fcmTokens);
    statusHandler.updateStatus(notification, result);
}
```

isEnabled()
사용자의 알림 수신 여부를
DB에서 조회

수신 거부 → return → 알림 상태 변경 없음 → 알림 테이블에 영원히 남음 → 다음 스케줄링에서 다시 조회

+) N+1 문제

로그로 확인

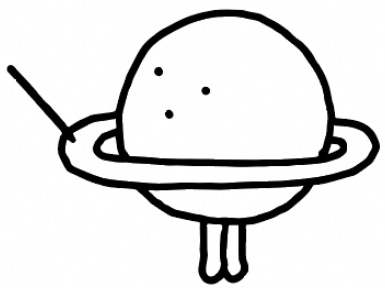
bombom-fcm

```
> {"@timestamp":"2026-09-22T17:04:12.643828168+09:00","message":"[ARTICLE] 처리할 알림 개수: 5173"}
> {"@timestamp":"2026-09-22T17:03:28.731026801+09:00","message":"[ARTICLE] 처리할 알림 개수: 5179"}
> {"@timestamp":"2026-09-22T17:02:39.364457225+09:00","message":"[ARTICLE] 처리할 알림 개수: 5191"}
> {"@timestamp":"2026-09-22T17:01:51.430162233+09:00","message":"[ARTICLE] 처리할 알림 개수: 5172"}
> {"@timestamp":"2026-09-22T17:01:00.731389356+09:00","message":"[ARTICLE] 처리할 알림 개수: 5173"}
> {"@timestamp":"2026-09-22T17:00:10.412757652+09:00","message":"[ARTICLE] 처리할 알림 개수: 5176"}
> {"@timestamp":"2026-09-22T16:59:21.450517845+09:00","message":"[ARTICLE] 처리할 알림 개수: 5179"}
> {"@timestamp":"2026-09-22T16:58:36.078083041+09:00","message":"[ARTICLE] 처리할 알림 개수: 5177"}
> {"@timestamp":"2026-09-22T16:57:50.294073966+09:00","message":"[ARTICLE] 처리할 알림 개수: 5179"}
> {"@timestamp":"2026-09-22T16:57:02.093188015+09:00","message":"[ARTICLE] 처리할 알림 개수: 5173"}
> {"@timestamp":"2026-09-22T16:56:15.430988333+09:00","message":"[ARTICLE] 처리할 알림 개수: 5177"}
> {"@timestamp":"2026-09-22T16:55:34.04913107+09:00","message":"[ARTICLE] 처리할 알림 개수: 5181"}
> {"@timestamp":"2026-09-22T16:54:41.772579317+09:00","message":"[ARTICLE] 처리할 알림 개수: 5181"}
> {"@timestamp":"2026-09-22T16:53:47.154267091+09:00","message":"[ARTICLE] 처리할 알림 개수: 5175"}
> {"@timestamp":"2026-09-22T16:52:58.097906965+09:00","message":"[ARTICLE] 처리할 알림 개수: 5180"}
> {"@timestamp":"2026-09-22T16:52:11.623307633+09:00","message":"[ARTICLE] 처리할 알림 개수: 5178"}
> {"@timestamp":"2026-09-22T16:51:27.608464053+09:00","message":"[ARTICLE] 처리할 알림 개수: 5180"}
> {"@timestamp":"2026-09-22T16:50:38.18814975+09:00","message":"[ARTICLE] 처리할 알림 개수: 5193"}
```

처리되지 않고 남아있는 알림이 약 5,100개

→ 30초 간격으로 조회 쿼리 5,100번

개선



개선 방향

1 수신 거부 알림은 삭제

현재: 바로 return
→ 스케줄러가 돌 때마다 영원히 "처리 대상"

알림을 켠다고 옛날 알림을 보내주는 건 이상함

어차피 로그를 남기고 있어 DB에 보관할 필요 없음
→ 바로 삭제하기로 결정

2 notification N + 1 쿼리 개선

memberId들을 IN 조건으로
FCM 토큰을 한 번에 조회

1번으로 처리 대상이 줄면 N + 1 오버헤드는 미미할 것
→ 1번 적용 후 지켜보기

→ 우선 1번만 적용

코드 개선 – 거부된 알림은 삭제 (코드 9줄 추가)

.../news/bombom/notification/service/NotificationProcessingService.java +1 Viewed

```
@@ -27,6 +27,7 @@ public <T extends Notification> void processNotification(T notification,
27  27          Long memberId = notification.getMemberId();
28  28          if (!notificationSettingService.isEnabled(memberId,
    category)) {
29  29              log.info("알림 수신 동의하지 않음: memberId={}, category={}",
    memberId, category);
30  +              statusHandler.handleRejected(notification);
30  31              return;
31  32          }
32  33
```

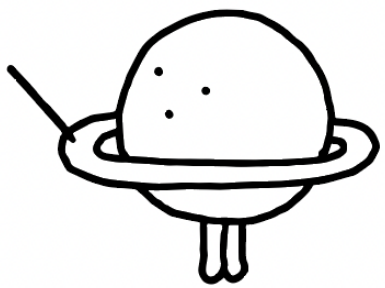
+1줄
수신 거부 시
handleRejected() 호출

.../bombom/article/service/ArticleArrivalNotificationStatusService.java +8 Viewed

```
@@ -53,6 +53,14 @@ public void markAsFailed(ArticleArrivalNotification notification, String reason)
53  53          moveToFailedTableIfExceeded(notification);
54  54      }
55  55
56  +      @Override
57  +      @Transactional
58  +      public void handleRejected(ArticleArrivalNotification
    notification) {
59  +          log.info("알림 수신 거부로 아티클 알림 삭제: notificationId={},
    memberId={}", notification.getId(),
60  +                  notification.getMemberId());
61  +          notificationRepository.delete(notification);
62  +      }
63  +
56  64      @Transactional
57  65      public void
    moveToFailedTableIfExceeded(ArticleArrivalNotification notification)
    {
58  66          if (!notification.shouldRetry()) {
```

+8줄
handleRejected() 추가
→ 삭제 로그 남기고
→ 알림 delete

결과



결과 – 첫 사이클

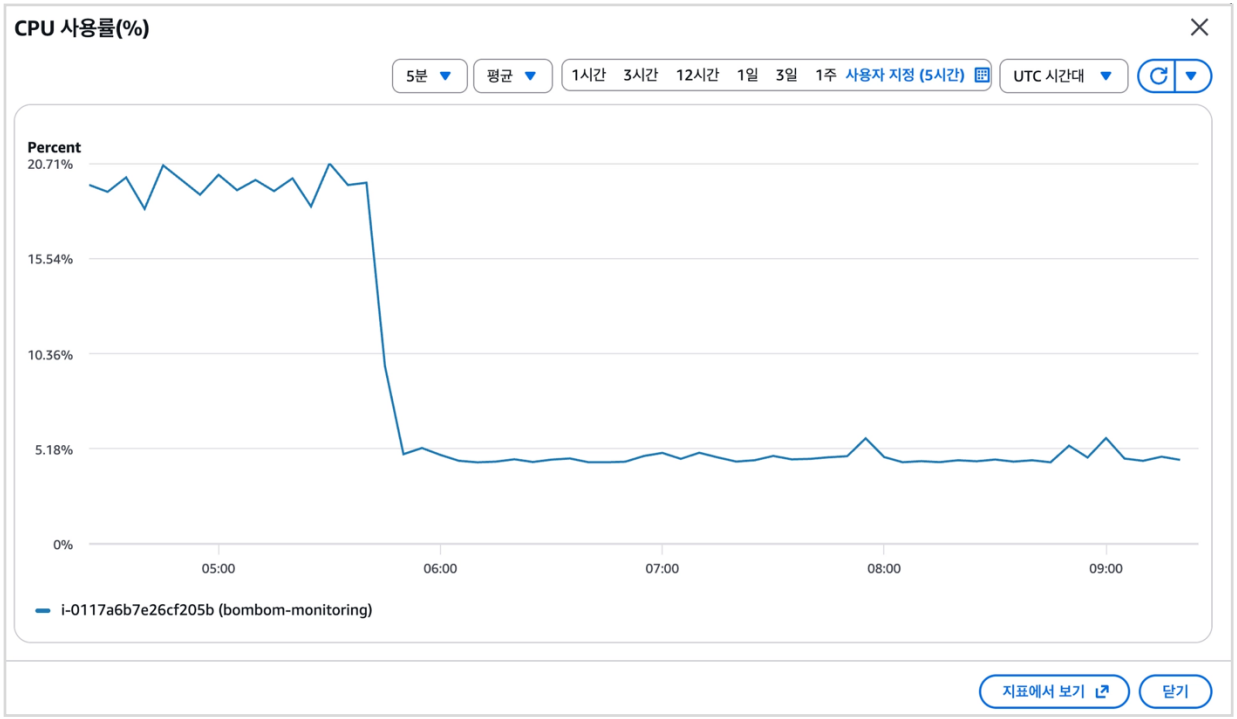
bombom-fcm

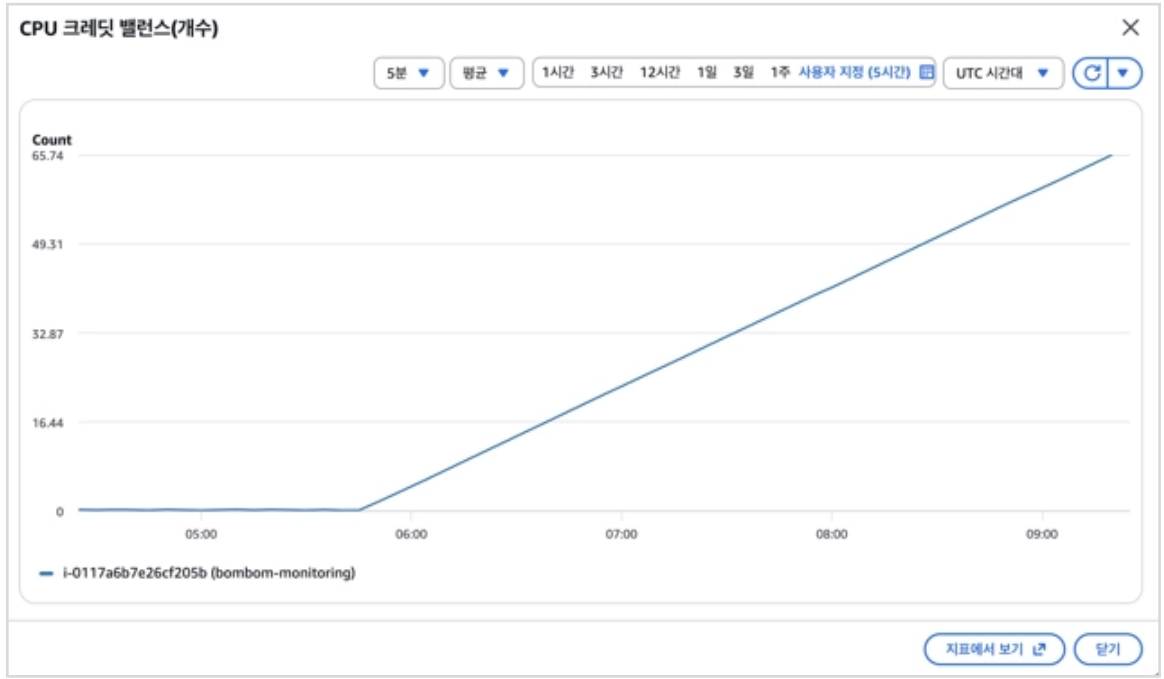
```
> {"@timestamp":"2026-10-01T14:46:06.94253057+09:00","message":"[ARTICLE] 처리할 알림 개수: 0"}
> {"@timestamp":"2026-10-01T14:42:15.794930147+09:00","message":"[ARTICLE] 처리할 알림 개수: 5599"}
> {"@timestamp":"2026-10-01T14:39:35.010045694+09:00","message":"[ARTICLE] 처리할 알림 개수: 5598"}
> {"@timestamp":"2026-10-01T14:38:39.474949464+09:00","message":"[ARTICLE] 처리할 알림 개수: 5598"}
> {"@timestamp":"2026-10-01T14:37:42.257260599+09:00","message":"[ARTICLE] 처리할 알림 개수: 5598"}
> {"@timestamp":"2026-10-01T14:36:39.302627049+09:00","message":"[ARTICLE] 처리할 알림 개수: 5598"}
> {"@timestamp":"2026-10-01T14:35:39.93982131+09:00","message":"[ARTICLE] 처리할 알림 개수: 5598"}
> {"@timestamp":"2026-10-01T14:34:44.185451853+09:00","message":"[ARTICLE] 처리할 알림 개수: 5598"}
> {"@timestamp":"2026-10-01T14:33:47.672877783+09:00","message":"[ARTICLE] 처리할 알림 개수: 5598"}
> {"@timestamp":"2026-10-01T14:32:53.062907288+09:00","message":"[ARTICLE] 처리할 알림 개수: 5598"}
> {"@timestamp":"2026-10-01T14:31:57.631210264+09:00","message":"[ARTICLE] 처리할 알림 개수: 5598"}
```

첫 사이클에서 수신 거부 알림 5,600개를 삭제 (약 4분 소요)

이후 처리 대상 알림은 0개

결과 – EC2 CPU 감소 & 크레딧 회복

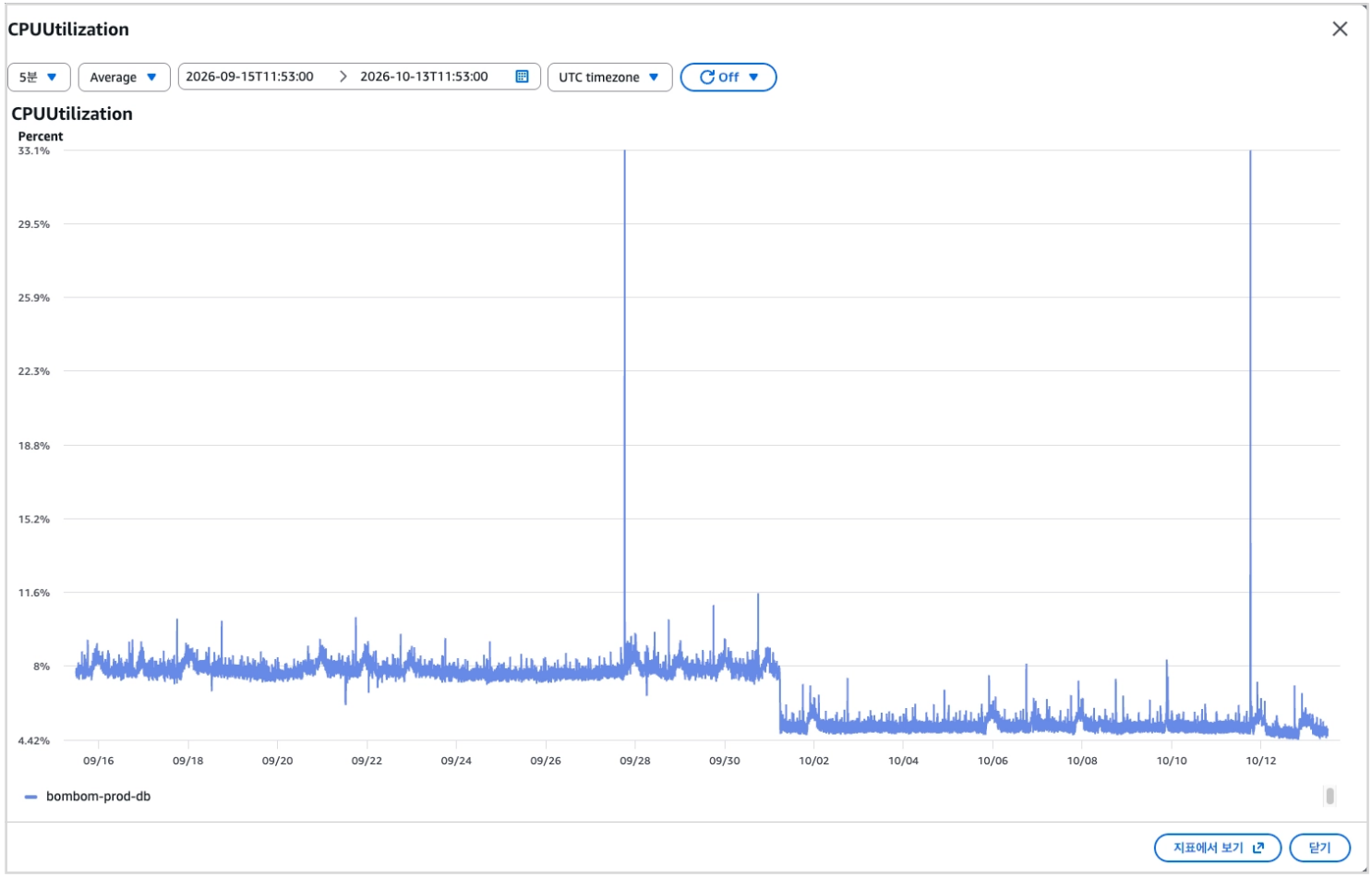




EC2 CPU 19.5% → 4.6%

CPU 크레딧 회복

결과 – DB CPU도 개선



DB CPU 8.7% → 5.4%

30초마다 나가던 쿼리 약 5,000개가 사라짐

결과 – JFR 비교

똑같이 1시간 측정한 뒤, 결과 보고서를 AI에 던져 비교

항목	Before (9/22 14:42~15:42)	After (10/1 18:26~19:26)	변화
전체 ExecutionSample	8,900	120	-98.7%
scheduling-1 샘플 (비율)	7,856 (88.3%)	49 (40.8%)	-99.4%
isEnabled 관련 샘플 (비율)	5,632 (63.3%)	5 (4.2%)	-99.9%
commit 관련 샘플 (비율)	904 (10.2%)	6 (5.0%)	-99.3%
AsyncAppender(로깅) 샘플	666	2	-99.7%
JVM CPU 평균 (user)	4.84%	0.90%	-81%
JVM CPU 평균 (system)	2.51%	0.14%	-94%
JVM CPU 평균 (user+system)	7.35%	1.04%	-86%
Machine Total CPU 평균 (JFR)	15.44%	4.11%	-73%

FCM 서버 JVM의 EC2 CPU 사용률 15.44% → 4.11%

JFR 전체 샘플 수 8,900 → 120

※ Before: 9/22 14:42~15:42 / After: 10/1 18:26~19:26 (각 1시간)

정리 – 숫자로 보는 개선

지표	개선 전	개선 후
모니터링 EC2 CPU	19.5%	4.6%
DB CPU	8.7%	5.4%
FCM 서버 JVM CPU (JFR)	15.44%	4.11%
처리되지 않고 쌓인 알림	약 5,100개	0개
CPU 크레딧	소진 · 스로틀링	회복

코드 9줄 추가로 스로틀링 해소

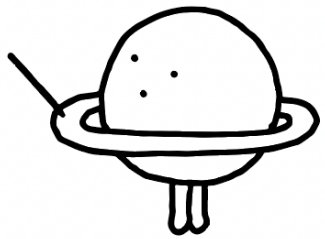
정리



처음부터 코드를 보면서 개선하려 했다면 힘들었을 것

관측 도구로 원인을 최대한 좁히고 해결하면

빠르게 해결할 수 있다



Q&A

